**SECTION 1: BASIC LEVEL**

**JDBC –**

1. **What is JDBC?**

**JDBC (Java Database Connectivity)** is a **Java API** that allows Java programs to **connect to a database**, send SQL queries, and retrieve results.

Think of JDBC as a **bridge between Java and databases**.

Example: If you want to fetch a student's marks from MySQL, you use JDBC.

2. **Why JDBC is required?**

JDBC is required to:

**Connect Java applications** to databases.

Execute **SQL queries** (SELECT, INSERT, UPDATE, DELETE) from Java.

Retrieve and process **database results**.

Provide **database independence** – works with MySQL, Oracle, PostgreSQL, etc., without changing Java code much.

3. **JDBC architecture (with diagram)?**

JDBC architecture has **4 layers**:

**Java Application** – Your Java code (like Main.java)

**JDBC API** – Provides interfaces like Connection, Statement, ResultSet

**JDBC Driver Manager** – Loads the correct driver for the database

**Database** – MySQL, Oracle, SQL Server, etc.

**Java Application**

**|**

**JDBC API (java.sql)**

**|**

**JDBC Driver**

**|**

**Database**

**Java app → JDBC API → Driver → Database**

4. **What are JDBC drivers?**

A **JDBC driver** is a **software component** that allows **Java to communicate with a specific database**.

- Example: MySQL Connector/J is a driver for MySQL.
- It converts Java calls into database-specific calls.

## 5. Types of JDBC drivers?

| Type | Description | Example |
|---|---|---|
| **Type 1** | JDBC-ODBC bridge driver | Obsolete now |
| **Type 2** | Native API driver (uses database client library) | Oracle driver |
| **Type 3** | Network Protocol driver (middleware server) | All Java apps to DB via middleware |
| **Type 4** | Pure Java driver (direct DB communication) | MySQL Connector/J, PostgreSQL JDBC |

## 6. Which JDBC driver is mostly used and why?

**Type 4 driver (Pure Java driver)**

**Why:**

- **Fast and platform-independent**
- **Direct communication with the database**
- **No extra libraries or middleware required**

## 7. Steps to connect Java application to database?

- Load driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- Create connection

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb", "root", "password");
```

- Create statement

```
Statement stmt = con.createStatement();
```

- Execute query

```
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
```

- Process result

```
while(rs.next()) {
    System.out.println(rs.getInt(1) + " " + rs.getString(2));
}
```

- Close resources

```
rs.close();
stmt.close();
con.close();
```

**8. What is DriverManager?**

- **DriverManager** is a **class** in `java.sql` package.
- It **manages database drivers** and establishes a connection using `getConnection()` method.

**9. What is Connection?**

Connection is an **interface** that:

Represents connection between Java app and database.

Note:- Without Connection, no DB operation is possible.

**Connection** represents a **session with the database**.

Provides methods to **create statements**, manage **transactions**, and **close connections**.

**10. What is Statement?**

Statement is an **interface** used to:

- Execute SQL queries

Using execute(),excecuteQuery().executeUpdate().

**Statement** is used to **execute SQL queries**.

Types:

1. Statement – For static SQL
2. PreparedStatement – For dynamic SQL with parameters
3. CallableStatement – For stored procedures

## 11. Difference between Statement and PreparedStatement?

| Statement | PreparedStatement |
|---|---|
| Simple SQL | Pre-compiled SQL |
| Slow | Faster |
| Not secure | Secure |
| SQL Injection possible | Prevents SQL Injection |
| Values hardcoded | Uses ? placeholders |

| Feature | Statement | PreparedStatement |
|---|---|---|
| SQL Type | Static SQL | Dynamic SQL |
| Pre-compilation | No | Yes (precompiled for speed) |
| SQL Injection | Vulnerable | Prevents SQL Injection |
| Usage | stmt.executeQuery("SELECT * FROM student"); | pstmt.setInt(1, 101); |

## 12. What is ResultSet?

ResultSet is an **object** that:

Stores data returned from SELECT query

Note:- It works like a **table of rows and columns**.

**ResultSet** holds the **data returned by a SELECT query**.

You can **iterate row by row** using rs.next().

```
while(rs.next()){
   System.out.println(rs.getInt("id") + " " + rs.getString("name"));
}
```

## 13. Types of ResultSet?

| ResultSet Type | Scrollable? | Explanation |
|---|---|---|
| TYPE_FORWARD_ONLY | No | Only forward |
| TYPE_SCROLL_INSENSITIVE | Yes | Forward & backward (no DB updates) |
| TYPE_SCROLL_SENSITIVE | Yes | Forward & backward + DB changes |

## 14. What is SQL Injection?

**SQL Injection** is a **security vulnerability** that occurs when a web application **directly uses user input to build SQL queries** without proper validation.

- The attacker can **inject malicious SQL code** into input fields like login forms, search boxes, or URL parameters.
- This allows them to **bypass authentication, read sensitive data, or even modify/delete database records**.

## 15. How does PreparedStatement prevent SQL Injection?

**PreparedStatement prevents SQL Injection because it sends the SQL and data separately. The input is treated strictly as data, so malicious SQL code cannot execute.**

```
String sql = "SELECT * FROM users WHERE username=? AND password=?";

PreparedStatement pstmt = con.prepareStatement(sql);

pstmt.setString(1, user);

pstmt.setString(2, pass);

ResultSet rs = pstmt.executeQuery();
```

? are **placeholders** for data

setString() tells the DB: "**this is data, not code**"

Even if hacker enters ' OR '1'='1, DB treats it as **a plain string**, not part of SQL logic

## 16. Difference between executeQuery() and executeUpdate()?

| executeQuery() | executeUpdate() |
|---|---|
| Used for SELECT | Used for INSERT, UPDATE, DELETE |
| Returns ResultSet | Returns int |

| executeQuery() | executeUpdate() |
| --- | --- |
| Fetch data | Modify data |

### 17. What is auto-commit?

Auto-commit means:

- Each SQL statement is **automatically saved** to DB

Default: **ON**

If OFF, you must use commit() manually.

```
con.setAutoCommit(false);
// execute SQL
con.commit();
```

### 18. How to close JDBC resources?

**With the help of close() method.**

Always close in this order:

1. ResultSet
2. Statement
3. Connection

```
if(rs != null) rs.close();
if(stmt != null) stmt.close();
if(con != null) con.close();
```

Use finally block or **try-with-resources** to avoid memory leak.

---

**Servlets –**

### 1. What is a Servlet?

A **Servlet** is a **Java program** that runs on a **server** and handles **client requests**, usually from a web browser. It can generate **dynamic content** (like HTML pages) and send it back to the client.

- Think of it as a **middleman** between the browser and the server.

- Example: When you log in to Facebook, a servlet can check your credentials and show your homepage.

## 2. Why Servlets are used?

Servlets are used to:

Handle client requests (GET, POST, etc.).

Generate dynamic web content (HTML, JSON, XML).

Access databases and perform business logic.

Maintain session information (like shopping carts).

Provide a **fast, scalable, and secure way** to create web applications.

## 3. Servlet architecture?

The Servlet architecture is based on the **client-server model**:

**Client**: Browser sends HTTP request.

**Web Server**: Receives request and forwards it to the servlet container.

**Servlet Container** (like Tomcat): Loads the servlet, manages its life cycle, and executes it.

**Servlet**: Processes request, interacts with database or business logic, generates response.

**Response**: Sent back through web server to the client.

Browser (Client)

|

| HTTP Request

v

Web Server

|

v

Servlet Container

|

v

Servlet

|

v

Response (HTML/JSON) --> Browser

## 4. What is a web server?

A **Web Server** is a software that:

- Handles **HTTP requests** from clients.
- Sends **static content** (like HTML, CSS, images) back to the client.
- Example: **Apache HTTP Server**, **Nginx**.

5. What is an application server?

An **Application Server** is software that:

- Runs **dynamic applications** like Servlets, JSP, EJB.
- Provides **business logic, database connectivity, transactions**, etc.
- Example: **Tomcat**, **JBoss/WildFly**, **GlassFish**.

## 6. Difference between web server and application server?

| Feature | Web Server | Application Server |
|---|---|---|
| Purpose | Serves **static content** | Serves **dynamic content** and apps |
| Supports | HTML, CSS, images | Servlets, JSP, EJB, database |
| Business Logic | No | Yes |
| Examples | Apache HTTP Server, Nginx | Tomcat, JBoss, GlassFish |

## 7. What is Tomcat?

**Tomcat** is a **Servlet container** (part of an application server) developed by Apache.

- It **runs Java Servlets and JSP pages**.
- Lightweight and widely used for **web applications**.

## 8. Servlet life cycle methods?

Servlet life cycle has **three main stages**:

1. **Initialization**
    - Method: init()
    - Called once when servlet is loaded. Used for setup tasks.
2. **Request Handling**
    - Methods: service(), doGet(), doPost()
    - Called **every time a client sends a request**.
3. **Termination**
    - Method: destroy()
    - Called once before servlet is removed from memory. Used to release resources.

## 9. Which method is called first in servlet life cycle?

**init()** is called **first** when the servlet is loaded.

## 10. Difference between doGet() and doPost()?

| Feature | doGet() | doPost() |
|---|---|---|
| Request Type | HTTP GET | HTTP POST |
| Data in URL | Sent as **URL parameters** | Sent in **request body** |
| Security | Less secure | More secure |
| Data Limit | Limited (about 2000 chars) | No practical limit |
| Example | Search query in Google | Login form submission |

## 11. What is HttpServlet?

**HttpServlet** is a **Java class** provided by `javax.servlet.http` package.

It is a **base class** to create HTTP-based servlets.

You extend this class and override `doGet()` or `doPost()` methods to handle requests.

```
import javax.servlet.*;
import javax.servlet.http.*;
```

```
import java.io.*;
public class HelloServlet extends HttpServlet {
   protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("<h1>Hello World!</h1>");
   }
}
```

## 12. What is web.xml?

`web.xml` is a **deployment descriptor** file for a Java web application.

It is located in `WEB-INF` folder.

It **maps URLs to servlets** and configures **initial parameters**.

**Example:**

```
<servlet>
   <servlet-name>HelloServlet</servlet-name>
   <servlet-class>HelloServlet</servlet-class>
</servlet>
<servlet-mapping>
   <servlet-name>HelloServlet</servlet-name>
   <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

## 13. What is URL mapping?

**URL Mapping** is the process of linking a **URL** to a **specific servlet**.

When the user requests a URL, the **mapped servlet** handles the request.

### 14. Types of URL mapping?

**Exact Match**

URL matches exactly the pattern in web.xml

Example: `/login`

**Prefix Match**

URL starts with a pattern

Example: `/user/*` handles `/user/profile`, `/user/settings`

**Extension Match**

URL ends with a specific extension

Example: `*.jsp` handles all JSP files

**Default Servlet**

`/` handles requests not mapped to any other servlet

### 15. What is deployment descriptor?

A **deployment descriptor** is a **configuration file** (`web.xml`) that tells the server how to deploy **web components** like servlets, filters, listeners.

Contains: servlet names, classes, URL mappings, initialization parameters.

---

**JSP –**

### 1. What is JSP?

**JSP (Java Server Pages)** is a **server-side technology** used to create **dynamic web pages** using **HTML + Java code**.

- JSP runs on the **server**
- Converts into a **Servlet internally**

- Used mainly for **presentation (UI layer)**

  Example: Showing user details dynamically on a webpage.

## 2. Difference between JSP and Servlet?

| Feature | JSP | Servlet |
|---|---|---|
| Type | Page-based | Java class |
| Code Style | HTML + Java | Pure Java |
| Used for | UI / Presentation | Business logic |
| Easy to write | Yes | Comparatively difficult |
| Converted into | Servlet | Already a servlet |

## 3. JSP life cycle?

JSP life cycle is **managed by the JSP container**.

**Steps:**

1. **Translation**
   JSP → Servlet (.java file generated)
2. **Compilation**
   Servlet .java → .class file
3. **Loading**
   Class is loaded into memory
4. **Initialization**
   jspInit() is called **once**
5. **Request Processing**
   _jspService() is called **for every request**
6. **Destruction**
   jspDestroy() is called before removal

JSP → Servlet → Class → jspInit()

→ _jspService() → Response

→ jspDestroy()

## 4. What are JSP implicit objects?

**Implicit objects** are **predefined objects** created automatically by the **JSP container**.

- No need to create them manually

- Available inside JSP pages
- Used for request handling, response, session, etc.

## 5. List all 9 implicit objects?

| Object | Purpose |
|---|---|
| request | Client request data |
| response | Send response to client |
| out | Output to browser |
| session | Session data |
| application | Application-level data |
| config | Servlet configuration |
| pageContext | Access all scopes |
| page | Current JSP instance |
| exception | Error handling |

## 6. What is scriptlet?

A **scriptlet** is used to write **Java code** inside JSP.

Syntax:

```
<%
   int a = 10;
   out.println(a);
%>
```

## 7. What is expression tag?

Expression tag is used to **print output directly** to the browser.

**Syntax:**

```
<%= a %>

No need to use out.println()
```

## 8. What is declaration tag?

Declaration tag is used to **declare variables and methods** at **class level**.

Syntax:

```
<%!
   int x = 5;
   int square(int n){
      return n*n;
   }
%>
```

**9. Difference between <% %> and <%= %>?**

| Tag | Purpose |
|---|---|
| <% %> | Java logic (scriptlet) |
| <%= %> | Output expression |

**10. What is JSP container?**

A **JSP container** is part of a **web/application server** that:

- Converts JSP to Servlet
- Compiles and executes JSP
- Manages JSP life cycle

**Example:**

- **Apache Tomcat**
- GlassFish

---

## 🔷 SECTION 2: INTERMEDIATE LEVEL

**JDBC –**

**1. Difference between PreparedStatement and CallableStatement?**

| Feature | PreparedStatement | CallableStatement |
|---|---|---|
| Used for | Normal SQL queries | Stored Procedures |
| SQL type | SELECT, INSERT, UPDATE | CALL procedure |
| Performance | Faster than Statement | Faster for complex logic |

| Feature | PreparedStatement | CallableStatement |
|---|---|---|
| Parameters | IN parameters | IN, OUT, INOUT |
| Stored in DB | No | Yes |

```
// PreparedStatement
String sql = "SELECT * FROM employee WHERE id=?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, 101);


// CallableStatement
CallableStatement cs = con.prepareCall("{call getEmployee(?)}");
cs.setInt(1, 101);
```

### PreparedStatement

- Used to execute **normal SQL queries** (SELECT, INSERT, UPDATE, DELETE)
- SQL is written inside Java program
- Accepts **only input (IN) parameters**
- Faster and safer than Statement
- Prevents SQL Injection

### CallableStatement

- Used to execute **stored procedures**
- Logic is already stored inside the database
- Supports **IN, OUT, and INOUT parameters**
- Used when business logic is inside DB
- Better for complex database operations

## 2. How to call stored procedures using JDBC?

A **Stored Procedure** is a **precompiled set of SQL statements** stored inside the database. JDBC provides a special interface called CallableStatement to call and execute these stored procedures from a Java application.

Ex:-

```
CREATE PROCEDURE getEmployee(IN empId INT)
BEGIN
  SELECT * FROM employee WHERE id = empId;
END;
```

```
CallableStatement cs = con.prepareCall("{call getEmployee(?)}");
cs.setInt(1, 101);
ResultSet rs = cs.executeQuery();
while(rs.next()){
   System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

### 3. What is batch processing in JDBC?

Batch processing means **executing multiple SQL statements together as a single batch** instead of executing them one by one.

All queries are sent to the database **at once**, reducing communication overhead.

```
PreparedStatement ps =
con.prepareStatement("INSERT INTO student VALUES(?,?)");
ps.setInt(1,1);
ps.setString(2,"A");
ps.addBatch();
ps.setInt(1,2);
ps.setString(2,"B");
ps.addBatch();
ps.executeBatch();
```

### 4. Advantages of batch processing?

- Faster execution
- Less network traffic
- Improved performance
- Efficient for bulk operations
- Reduces database load
- Mostly used in **bulk insert/update/delete operations**

### 5. What is connection pooling?

Connection pooling is a technique where **multiple database connections are created in advance and stored in a pool**.

Instead of creating a new connection every time, the application **reuses existing connections**.

### 6. Why connection pooling is important?

- Creating a DB connection is time-consuming
- Reduces memory and CPU usage
- Improves application performance
- Handles multiple users efficiently
- Essential for enterprise applications

Without pooling → slow application

With pooling → fast and scalable application

### 7. How JDBC handles transactions?

JDBC manages transactions using:

Auto-commit mode

Manual commit and rollback

By default, JDBC uses **auto-commit**, meaning each query is saved immediately.

To control transactions manually, auto-commit is disabled.

### 8. What is commit and rollback?

**Commit**

Saves all changes permanently to the database

Makes transaction successful

**Rollback**

Cancels changes made during a transaction

Restores database to previous state

### 9. What happens if commit is not called?

If auto-commit is disabled and `commit()` is not called:

Changes are **not saved**

On error or crash, data is lost

JDBC automatically performs rollback

### 10. What is savepoint?

A savepoint is a **checkpoint inside a transaction**.

It allows rollback to a **specific point**, not the entire transaction.

### 11. What is database metadata?

Database metadata means **data about the database**.

**It includes:**

Database name

Tables

Columns

Data types

Constraints

Database version

### 12. What is DatabaseMetaData?

`DatabaseMetaData` is a JDBC interface used to **retrieve database-level information**.

It provides:

Database product name

Driver details

Supported features

List of tables and schemas

### 13. What is ResultSetMetaData?

`ResultSetMetaData` provides **information about columns returned by a query**.

It tells:

Number of columns

Column names

Data types

Column sizes

*** Used when column structure is unknown at runtime.**

### 14. Difference between forward-only and scrollable ResultSet?

**Forward-only ResultSet**

Can move only forward

Faster performance

Default ResultSet type

**Scrollable ResultSet**

Can move forward and backward

Allows random access

Slightly slower

### 15. How to insert image/file using JDBC?

Images and files are stored in the database as **BLOB (Binary Large Object)**.

JDBC allows binary data to be:

Read as stream

Sent to database

Stored safely in BLOB column

**Used for photos, PDFs, documents, etc.**

---

**Servlets –**

### 1. Difference between ServletConfig and ServletContext?

ServletConfig

Belongs to **one specific servlet**

Used to read **servlet-specific parameters**

Created once per servlet

## ServletContext

Belongs to **entire web application**

Shared among **all servlets**

Used for application-wide data

**.xml**

```
<servlet>
  <servlet-name>Test</servlet-name>
  <init-param>
    <param-name>admin</param-name>
    <param-value>Raghu</param-value>
  </init-param>
</servlet>
```

**Java:-**

```
ServletConfig config = getServletConfig();
String name = config.getInitParameter("admin");

ServletContext context = getServletContext();
String app = context.getInitParameter("appName");
```

### 2. What is request dispatching?

Request dispatching means **sending the same request from one servlet to another resource** (Servlet/JSP/HTML).

Used to:

Reuse code

Separate logic

## Types

`forward()`

`include()`

### 3. Difference between forward() and sendRedirect()?

| Feature | forward() | sendRedirect() |
|---|---|---|
| Control | Server | Client |
| Request object | Same | New |
| URL change | No | Yes |
| Speed | Fast | Slower |

### 4. What is session?

A session is a **way to store user data across multiple requests**.

Since HTTP is stateless, sessions help remember:

Login info

User preferences

### 5. Types of session tracking?

Cookies

URL Rewriting

Hidden Fields

HttpSession

### 6. Cookies vs HttpSession?

| Feature | Cookies | HttpSession |
|---|---|---|
| Stored at | Client | Server |
| Security | Less | More |
| Size | Limited | Large |
| Lifetime | Configurable | Until timeout |

### 7. How cookies work?

Server creates cookie

Cookie sent to browser

Browser stores it

Browser sends it back with every request

```
Cookie c = new Cookie("user","Raghu");

response.addCookie(c);
```

### 8. Difference between hidden fields and URL rewriting?

| Feature | Hidden Fields | URL Rewriting |
|---|---|---|
| Visibility | Hidden in HTML | Visible in URL |
| Data type | Small data | Small data |
| Security | Low | Very low |

```
<input type="hidden" name="id" value="10">

response.sendRedirect("home.jsp?id=10");
```

### 9. What is filter?

A filter is a **component that intercepts requests and responses** before they reach a servlet.

Used for:

Validation

Logging

Authentication

### 10. Use cases of filters?

1. Login authentication
2. Request logging
3. Input validation
4. Compression
5. CORS handling

### 11. What is listener?

A listener is a **component that listens to events** in web application lifecycle.

Triggered automatically by container.

### 12. Types of listeners?

1.ServletContextListener

2. HttpSessionListener

3. ServletRequestListener

4. Attribute Listeners

### 13. What is MVC architecture?

MVC = **Model + View + Controller**

Model → Business logic & data

View → UI (JSP/HTML)

Controller → Handles request (Servlet)

### 14. How servlet fits in MVC?

Servlet acts as **Controller**.

Flow:

1. Client request
2. Servlet processes logic
3. Calls Model
4. Forwards to JSP (View)

### 15. What is @WebServlet annotation?

`@WebServlet` is used to **configure servlet without web.xml**.

It defines:

Servlet name

URL pattern

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
}
```

---

**JSP –**

### 1. Difference between JSP include and directive include?

## JSP Include (`<jsp:include>`)

- Executed **at runtime**
- Included file is processed **every request**
- Changes reflect **immediately**
- Uses request and response objects

## Directive Include (`<%@ include %>`)

- Executed **at translation time**
- Included once when JSP is converted to servlet
- Faster execution
- Changes require **server restart**

### 2. What is JSTL?

JSTL (JSP Standard Tag Library) is a **collection of predefined tags** used to perform common tasks like:

- Looping
- Conditions
- Formatting
- Database access
- **It removes Java code from JSP pages.**

3. **Why JSTL is used?**

Avoids Java code in JSP

Improves readability

Easy to maintain

Follows MVC architecture

Reduces errors

 Makes JSP pages **clean and professional**

4. **Types of JSTL tags?**

- **Core Tags** (`c`)

  if, forEach, choose

- **Formatting Tags** (`fmt`)

  Date, number formatting

- **SQL Tags** (`sql`)

  DB operations (not recommended)

- **XML Tags** (`x`)

  XML processing

- **Function Tags** (`fn`)

  String operations

5. **What is EL (Expression Language)?**

EL is a **simple expression language** used to **access data** stored in:

request

session

application

page scope

6. **Advantages of EL?**

No Java syntax

- Easy to read
- Less error-prone
- Faster development
- Supports implicit objects

### 7. How to access session attributes in JSP?

**${sessionScope.username}**

### 8. Difference between JSP and HTML?

| Feature | JSP | HTML |
|---|---|---|
| Dynamic | Yes | No |
| Java support | Yes | No |
| Server-side | Yes | No |
| Compilation | Converted to servlet | Not compiled |

### 9. What is custom tag?

A custom tag is a **user-defined JSP tag** created to:

Encapsulate reusable logic

Replace scriptlets

Improve code reuse

 Looks like HTML but executes Java logic.

### 10. Why scriptlets are discouraged?

Scriptlets (<% %>) are discouraged because:

- Mix Java & HTML
- Hard to read
- Hard to maintain
- Break MVC pattern

- Difficult for designers

---

## ◈ SECTION 3: ADVANCED LEVEL

**JDBC –**

### 1. How JDBC works internally?

## Internal Flow of JDBC

Java application sends SQL

JDBC API calls JDBC Drive

Driver converts Java calls → DB-specific protocol

Database executes SQL

Result is sent back

Driver converts DB result → Java objects.

### 2. What happens internally when executeQuery() is called?

When `executeQuery()` is called:

1. SQL is validated
2. JDBC driver sends SQL to DB
3. DB parses and optimizes query
4. DB executes query
5. Result stored in DB cursor
6. ResultSet object created
7. Rows fetched one by one

### 3. How connection pooling works internally?

## Internal Steps

Pool creates fixed number of connections at startup

Client requests a connection

Pool gives an **idle connection**

Client uses connection

Client closes connection

Connection returned to pool (not destroyed)

 **Pool manages:**

Max connections

Idle timeout

Validation

### 4. Difference between optimistic and pessimistic locking?

| Feature | Optimistic Locking | Pessimistic Locking |
|---|---|---|
| Lock timing | At update time | At read time |
| Blocking | No | Yes |
| Performance | High | Low |
| Use case | Read-heavy apps | Write-heavy apps |

### 5. How JDBC handles concurrency?

JDBC handles concurrency using:

Transactions

Isolation levels

Locks

Connection pooling

## Isolation Levels

READ_UNCOMMITTED

READ_COMMITTED

REPEATABLE_READ

SERIALIZABLE

### 6. How to improve JDBC performance?

## Best Techniques

Use PreparedStatement

Enable connection pooling

Use batch processing

Fetch only required columns

Use proper indexes

Avoid auto-commit

### 7. What are JDBC best practices?

Close resources in `finally`

Use PreparedStatement

Avoid hard-coded SQL

Use connection pooling

Handle exceptions properly

Use transactions carefully

### 8. How to handle large data using JDBC?

## Techniques

Use streaming (`setFetchSize`)

Use BLOB / CLOB for files

Use pagination

Avoid loading all rows at once

### 9. How to avoid memory leaks in JDBC?

## Common Causes

Not closing ResultSet

Not closing Statement

Not closing Connection

## Prevention

Use try-with-resources

Close in reverse order

Return connection to pool

10. **How transactions work in distributed systems?**

## Distributed Transaction

Transaction involving **multiple databases / services**

## Managed Using:

Two-Phase Commit (2PC)

## Phase 1: Prepare

Ask all DBs “Are you ready?”

## Phase 2: Commit/Rollback

If all say YES → commit

If any say NO → rollback

---

**Servlets –**

1. **How servlet container manages threads?**

A servlet container (Tomcat, Jetty, etc.) uses a **thread pool**.

## Internal Working

Container starts and creates **one servlet instance**

Creates a **pool of threads**

Each client request is assigned a **separate thread**

Thread executes `service()` → `doGet()` / `doPost()`

Thread returns to pool after completion

2. **Is servlet thread-safe?**

No, servlets are NOT thread-safe by default

Because:

Multiple threads access the **same servlet object**

Shared variables can be modified simultaneously

### 3. How to make servlet thread-safe?

## Methods

1. **Avoid instance variables**
2. Use **local variables** (thread-safe)
3. Use **synchronization** (with caution)
4. Use **immutable objects**
5. Store user data in **request/session**, not servlet fields

### 4. Difference between SingleThreadModel and synchronization?

| Feature | SingleThreadModel | Synchronization |
|---|---|---|
| Threads | One at a time | Multiple threads |
| Performance | Poor | Better |
| Status | Deprecated | Recommended |
| Scalability | Low | High |

### 5. What is load-on-startup?

load-on-startup tells the container **when to load a servlet**.

- 0 or positive → Load when server starts
- Negative or missing → Load on first request

### 6. How session replication works?

Session replication is used in **clustered environments**.

## Internal Flow

User logs in

Session created on Server A

Session data copied to Server B

If Server A fails → Server B continues session

## Types

In-memory replication

Database-based

Sticky sessions

### 7. How to secure a servlet?

## Techniques

- Authentication
- Authorization
- HTTPS (SSL)
- Filters
- Input validation
- CSRF protection

### 8. How authentication works in servlets?

## Steps

1. User sends credentials
2. Container validates user
3. Creates security context
4. Grants or denies access
5. Session maintained

## Types

- BASIC
- FORM
- DIGEST
- CLIENT-CERT

### 9. Difference between container-managed and programmatic security?

| Feature | Container-Managed | Programmatic |
|---|---|---|
| Who handles | Container | Developer |
| Configuration | web.xml / annotations | Java code |
| Flexibility | Less | More |
| Complexity | Low | High |

### 10. How filters work internally?

## Internal Flow

Request comes to container

Filter intercepts request

doFilter() executed

Request forwarded to servlet

Response passes back through filter

**Chain Concept**

Client → Filter → Servlet → Filter → Client

### 11. What is CSRF?

CSRF (Cross-Site Request Forgery) is an attack where:

User is authenticated

Attacker tricks browser into sending a request

Action happens **without user consent**

## Example

Hidden malicious form auto-submits a request.

### 12. How to prevent CSRF in servlets?

## Techniques

- CSRF tokens
- Synchronizer token pattern
- SameSite cookies
- POST instead of GET
- Referer validation

## Concept

Server generates a **random token** and validates it.

### 13. How servlet handles concurrent requests?

One servlet instance

Multiple threads

Each thread handles one request

Shared memory → risk

Local data → safe

### 14. What happens if multiple users hit same servlet?

All users share **same servlet object**

Each user gets **different thread**

Request/response objects are separate

Instance variables are shared

### 15. How to optimize servlet performance?

## Best Practices

- Use connection pooling
- Avoid synchronization blocks
- Use filters wisely
- Cache static data
- Use load-on-startup
- Avoid heavy logic in servlet
- Use async processing

---

**JSP –**

### 1. How JSP is converted into Servlet?

JSP is **not executed directly**. Internally, the container **converts JSP into a Servlet**.

## Internal Flow

Client requests JSP

Container checks if servlet exists

If not:

JSP → Java Servlet code

Java code → `.class` file

Servlet lifecycle starts

Response sent to client

2. **Explain JSP translation and compilation phase?**

## Translation Phase

JSP is converted into a **Servlet (.java file)**

JSP tags, EL, JSTL → Java code

## Compilation Phase

Generated `.java` file is compiled into `.class`

## When It Happens?

First request

Or when JSP is modified

3. **Why JSP is slower than servlet initially?**

Because JSP has **extra steps**:

- Translation
- Compilation
- Class loading

Servlets are already compiled.

4. **How to improve JSP performance?**

## Best Practices

- Precompile JSPs
- Avoid scriptlets
- Use JSTL + EL
- Cache static data
- Minimize includes
- Use proper scopes
- Avoid heavy logic in JSP

### 5. How EL is evaluated internally?

## EL Evaluation Flow

Container parses EL expression

Identifies scope (page, request, session, application)

Searches attribute

Calls getter method

Converts value to string

Displays output

### 6. JSP vs Thymeleaf vs FreeMarker?

| Feature | JSP | Thymeleaf | FreeMarker |
|---|---|---|---|
| Type | Servlet-based | Template engine | Template engine |
| MVC fit | Medium | Excellent | Excellent |
| HTML friendly | NO | YES | YES |
| Spring support | Limited | Excellent | Excellent |
| Modern usage | Old | Popular | Popular |

### 7. JSP security issues?

## Common Issues

XSS attacks

Session hijacking

Sensitive data exposure

Scriptlet misuse

Improper validation

### 8. How to prevent XSS in JSP?

Use `<c:out>` instead of `<%= %>`

Use EL escaping

Validate user input

Encode output

Use Content Security Policy (CSP)

### 9. What happens if JSP throws exception?

## Internal Process

i. Exception occurs
ii. Container stops JSP execution
iii. Checks for error page mapping
iv. Forwards to error page
v. Displays user-friendly message

### 10. How error pages work in JSP?

Configuration Methods

*JSP level;-*

```
<%@ page errorPage="error.jsp" %>
```

*Error page JSP*

```
<%@ page isErrorPage="true" %>
```

***web.xml mapping***

```
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/error.jsp</location>
</error-page>
```

---

**🔷 SECTION 4:**

**💡 JDBC + Servlets + JSP**

### 1. Design login system using JDBC + Servlet + JSP

A login system is used to **authenticate users** before allowing access to secured pages.

## Conceptually:

**JSP** is used to collect user input (username & password).

**Servlet** acts as a controller that receives the request.

**JDBC** communicates with the database to validate credentials.

### Why this design?

Separation of concerns

Easy maintenance

Secure handling of data

### Real-time Example:

Almost all web apps (Gmail, Facebook, Banking apps) follow this flow.

---

**2. How do you validate user input securely?**

Input validation ensures that **only correct and safe data** enters the system.

### Why validation is important?

Prevents SQL Injection

Prevents XSS attacks

Avoids application crashes

### Levels of Validation:

**Client-side** (JavaScript) – Fast but not secure

**Server-side** (Servlet) – Mandatory and secure

**3. How do you prevent SQL Injection?**

SQL Injection is an attack where malicious SQL code is injected into input fields.

### Why it happens?

When user input is directly concatenated into SQL queries

## Prevention Techniques:

Use `PreparedStatement`

Never build dynamic SQL using user input

Validate input

### 4. How do you manage session timeout?

Session timeout defines **how long a user stays logged in without activity**.

## Why needed?

Security

Prevent unauthorized access

Resource management

## Common Uses:

Banking apps → 5–10 minutes

Shopping apps → 30 minutes

### 5. How to upload file using servlet?

File upload allows users to send files (images, PDFs, resumes) to the server.

## Concepts Involved:

Multipart HTTP request

Servlet API (`Part` interface)

Server storage

## Real-time Uses:

Resume upload

Profile picture

Documents submission

### 6. How to download file using servlet?

File download sends files from server to client browser.

## Key Concepts:

HTTP response headers

MIME types

Output stream

## Real-time Uses:

Invoice download

Report export

Certificate download

### 7. How pagination is implemented using JDBC?

Pagination divides large datasets into **smaller chunks (pages)**.

## Why pagination?

Improves performance

Reduces memory usage

Better user experience

## How it works:

Fetch limited records per request

Use offset to track pages

### 8. How to implement logout?

Logout means **ending the user session**.

## Key Concept:

Session invalidation

## Why invalidate session?

Removes all stored user data

Prevents session reuse

Enhances security

9. **How to implement role-based access?**

Role-based access control (RBAC) restricts pages based on user roles.

### Common Roles:

ADMIN

USER

MANAGER

### How it works:

Role stored in session

Each request checks role

Access granted or denied

### Real-time Example:

Admin → User management

User → Profile only

10. **How to handle 10,000 concurrent users?**

Handling high traffic means **efficient resource usage**.

### Key Challenges:

Database overload

Thread exhaustion

Memory usage

### Solutions:

Connection pooling

Stateless servlets

Caching

Load balancing

### Important Fact:

Servlets are multi-threaded by default.

### 11. How to implement MVC using Servlet + JSP?

MVC separates application into **three logical layers**.

### MVC Layers:

**Model** → Business logic & DB

**View** → JSP UI

**Controller** → Servlet

### Benefits:

Clean code

Easy testing

Better scalability

### 12. How to improve performance without using Spring?

Even without frameworks, performance can be optimized.

### Techniques:

Connection pooling

Caching frequently used data

Proper indexing in DB

Avoid heavy logic in JSP

### 13. How to debug production issues?

Production debugging means finding issues **without stopping the app**.

## Techniques:

- Log analysis
- Exception tracking
- Reproducing issues in test environment
- Monitoring tools

### 14. How to handle DB failure?

Database failure may occur due to:

- Network issues
- DB crash
- Connection timeout

## Handling Strategy:

- Use try-catch
- Show user-friendly error
- Retry logic
- Fallback mechanism

### 15. How to log errors in servlet applications?

Logging captures **application behavior and errors**.

## Why logging?

- Debugging
- Monitoring
- Auditing

## Types of Logs:

- INFO

DEBUG

ERROR

---